

UGV-UAV

Jnanesh Sathisha Karmar
Vivek K Kumar

February 23, 2026

Contents

1	Introduction	3
1	Motivation & Objective	3
2	Unmanned Aerial Vehicle (UAV)	4
1	The Brain: ArduPilot Flight Controller	4
1.1	How to load firmware	4
1.2	Mission Planner	4
3	Dabble	6
1	Component Architecture with Dabble	6
2	Controlling Ground Vehicle with Dabble	6
4	FLYSKY Remote	8
1	Component Architecture in UGV (FlySky)	9
5	Operating UGV through ArduPilot	10
1	Integration with L298N Motor Drivers	10
1.1	Setup	11
1.2	Result	11

1 Introduction

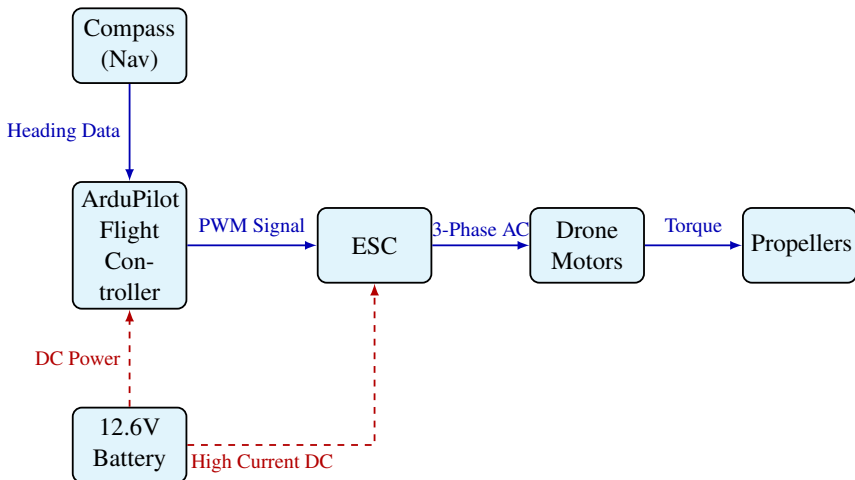
1 Motivation & Objective

To integrate UAV architecture into UGV and have a unified control system for both ecosystems. This is efficient, and also enables for the testing of UAVs through UGVs, as effectively UAVs only have an extra dimension.

2 Unmanned Aerial Vehicle (UAV)

1 The Brain: ArduPilot Flight Controller

ArduPilot is a versatile autopilot system for **Multicopters, Rovers, Boats, and Planes**. It reads sensors (Gyro, GPS) to controls motors. One of the most important advantages of this is that it handles both high-speed flight stabilization and precise ground steering.



1.1 How to load firmware

We do not have to write C++ code from scratch to run the microcontroller. Pre-compiled firmware is available in the ArduPilot website, from which Flash specific stacks (e.g., ArduCopter/ArduRover) can be flashed into ArduPilot via Mission Planner. Mission Planner can also be used to fix the initial parameters (e.g., Radio calibration, ESC calibration) to get the UAV ready for flight.

1.2 Mission Planner

Mission Planner is a free, open-source Ground Control Station (GCS) software used to configure, monitor, and control autonomous vehicles such as drones, planes,

rovers, and boats that run on ArduPilot firmware. It provides a graphical interface to communicate with the flight controller through USB or telemetry radios. Mission Planner allows users to upload flight missions, set waypoints, tune parameters, calibrate sensors, and monitor real-time flight data such as altitude, speed, GPS position, and battery status.

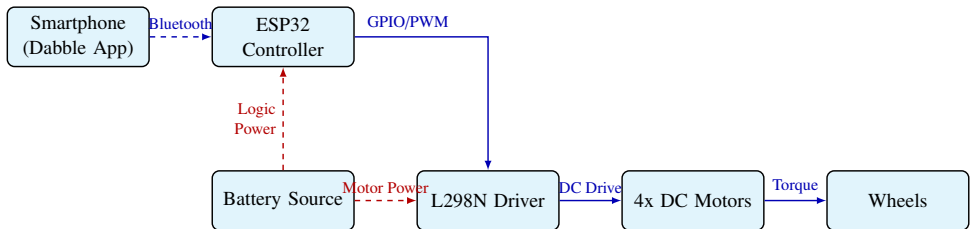


Figure 0.1: Mission Planner interface

3 Dabble

Dabble is a mobile application developed by STEMpedia that enables smartphones to function as wireless controllers for microcontroller-based systems such as Arduino, ESP32, and other embedded platforms. It uses Bluetooth communication to connect the smartphone with the hardware, allowing users to control, monitor, and interact with electronic projects in real time. Dabble provides a set of pre-built modules that simplify interfacing between hardware and smartphone sensors, eliminating the need to design complex mobile applications from scratch. Through its graphical interface, users can access smartphone features such as buttons, sensors, camera input, and motion detection, and transmit that data directly to the microcontroller.

1 Component Architecture with Dabble



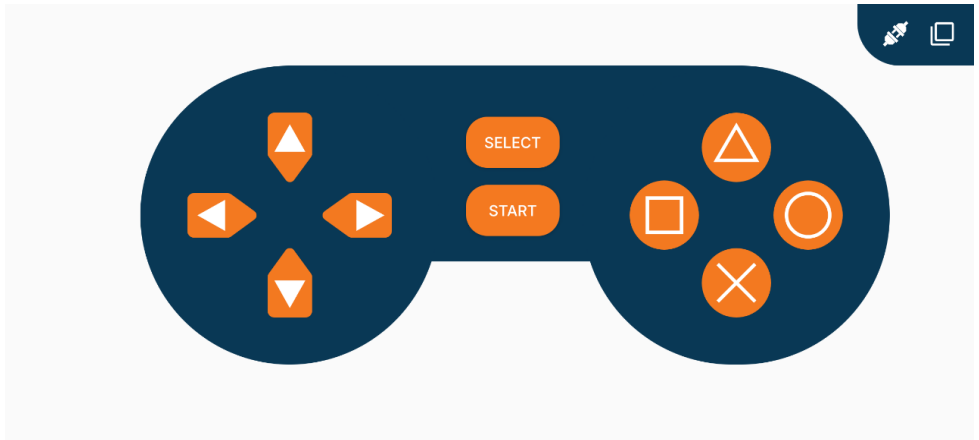
2 Controlling Ground Vehicle with Dabble

Setup & Connection:

1. Install **Dabble** app (Android).
2. Enable Bluetooth on your phone.
3. Open the app and select the **Gamepad** module.
4. Tap the **Plug Icon** (Connect) at the top right.
5. Select your ESP32 (e.g., "JNANU").

Controls:

- Use the **Digital Mode** (Arrow keys).
- **Up/Down:** Move Forward/Backward.
- **Left/Right:** Turn (Differential Drive).



Dabble Gamepad Interface

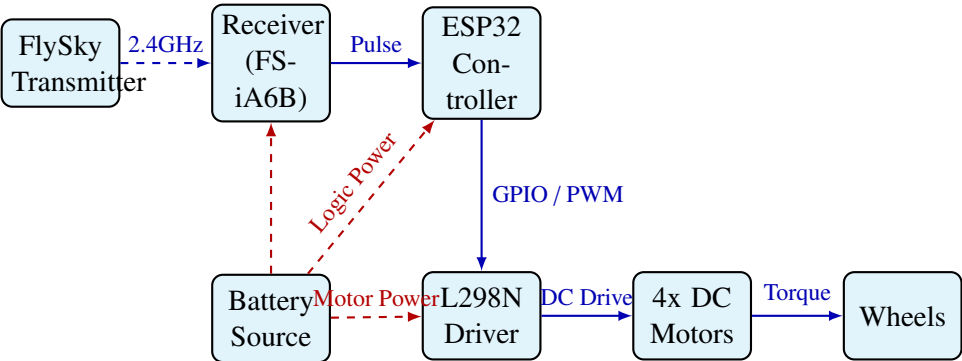
4 FLYSKY Remote

The Flysky FS-i6 is a 2.4 GHz wireless radio transmitter used for remote control of electronic systems such as drones, robots, RC cars, airplanes, and embedded systems. It is part of the Flysky radio control ecosystem and works together with compatible receivers (such as FS-iA6 or FS-iA6B) to transmit user commands wirelessly to a microcontroller or motor control system. The FS-i6 converts physical inputs from joysticks, switches, and knobs into digital signals, which are transmitted using the AFHDS 2A (Automatic Frequency Hopping Digital System) protocol. The receiver connected to the robot or vehicle interprets these signals and sends corresponding control signals to motors, servos, or microcontrollers.



Figure 5.1: flysky remote

1 Component Architecture in UGV (FlySky)



5 Operating UGV through ArduPilot

1 Integration with L298N Motor Drivers

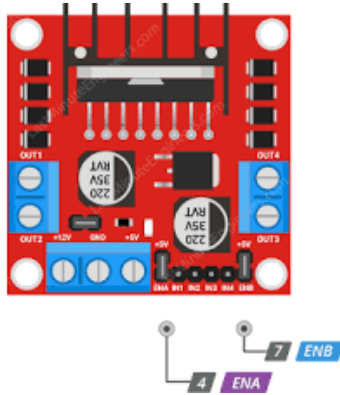


Figure 5.1: L298N

The system architecture follows a sequential control flow from the APM 2.8 flight controller, through an ESP32 microcontroller, to an L298N motor driver, which ultimately powers the DC motors. A direct connection between the APM and the motor driver is unfeasible due to a fundamental signal mismatch. Specifically, the APM 2.8 outputs standard RC servo PWM pulses ranging from $1000\ \mu\text{s}$ to $2000\ \mu\text{s}$, whereas the L298N driver requires logic-level directional inputs (HIGH/LOW) and standard duty-cycle PWM (0–100%) to regulate speed. To bridge this gap, the ESP32 functions as a dedicated signal translator. It continuously reads the incoming pulse widths from the APM’s throttle and steering outputs and mathematically maps these values to the required motor driver formats. For example, the ESP32 is programmed to interpret a signal of $1000\text{--}1450\ \mu\text{s}$ as a reverse command, $1450\text{--}1550\ \mu\text{s}$ as a neutral deadband for stopping or braking, and $1550\text{--}2000\ \mu\text{s}$ as forward motion. Based on this real-time mapping, the ESP32 drives the appropriate pins on the L298N to achieve precise locomotion.

1.1 Setup

1. Install ArduRover v2.5.1 firmware in mission planner for APM 2.8
2. Calibrate radio, compass, GPS, accelerometer and servo output in Mission Planner
3. Wire APM 2.8 as follows:
 - Inputs: Connect radio receiver and GPS/Compass module. Also power it up through the 5V L298N motor driver line.
 - Outputs: Connect signal lines of 1 and 3 to ESP32.
4. Upload the code into the ESP32. The code is available here

<https://github.com/Vivek-Kumar-1907/uavugv/blob/main/codes/main.ino>

1.2 Result

We have hence successfully translated the entire UAV architecture into UGV architecture.